

AGENTIC AI REFERENCE ARCHITECTURE

Project Sentinel

A spatial & telemetry monitoring system for autonomous systems — combining a real-time FastAPI backend, the Model Context Protocol, a LangGraph multi-agent workflow, and RAG-grounded diagnosis.

project_sentinel · github.com/gowthamde24/project_sentinel

FastAPI

MCP

LangGraph

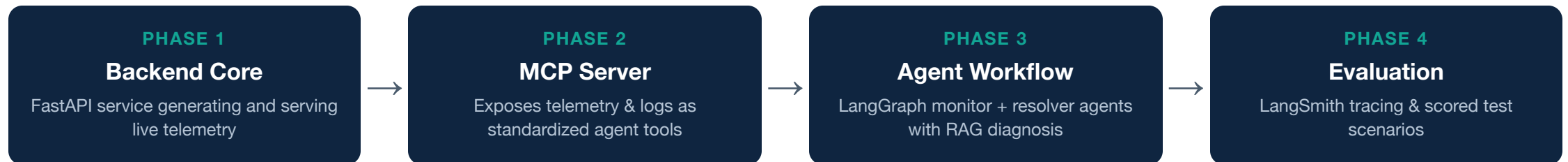
Pinecone RAG

What Project Sentinel Does

- **Simulates** live hardware and spatial telemetry for an autonomous system — CPU/RAM/power, IMU orientation, and SLAM positional drift.
- **Monitors** that telemetry continuously and flags anomalies the moment they occur, using deterministic, explainable rules.
- **Diagnoses** root cause automatically: an AI agent retrieves the relevant technical documentation and generates a grounded explanation plus a recommended fix.
- **Demonstrates** a complete, production-shaped enterprise AI stack — API layer, standardized tool access, multi-agent orchestration, and evaluation — built incrementally across four phases.

"Deterministic where possible, AI where necessary" — anomaly detection is rule-based and free; the LLM is invoked only when there is something worth diagnosing.

Four Phases, One Pipeline



Each phase is independently runnable and testable — the backend works stand-alone, the MCP server degrades gracefully if the backend is offline, and the agent workflow only calls the LLM when telemetry is actually anomalous.

PHASE 1

Backend Core

A FastAPI service that continuously simulates realistic hardware and spatial telemetry in the background, without ever blocking the API thread.

Real-Time Telemetry Service

- An async background task generates a new telemetry reading **every second**, running alongside the API without blocking requests.
- Each reading models three domains: **hardware** (CPU/RAM/power draw), **IMU** (acceleration & orientation), and **drift** (cumulative SLAM positional error).
- Cumulative drift above a threshold automatically sets a **SLAM failure** flag, then resets — simulating a recovery cycle.
- The last 100 readings are kept in a rolling in-memory history buffer.

Endpoints

GET /health	Liveness check
GET /telemetry/current	Latest reading
GET /telemetry/history	Recent history, capped by limit

PHASE 2

MCP Server

Exposes the backend's telemetry and logs as standardized tools any Model Context Protocol-compatible agent can call — without giving the agent direct code access.

Standardized Tool Access

get_current_telemetry

Fetches live data from the FastAPI backend over HTTP. Returns a clear error object if the backend is unreachable, rather than failing silently.

query_system_logs

Performs a case-insensitive keyword search over the local system log file (e.g. "ERROR", "SLAM") and returns matching lines, capped by a max count.

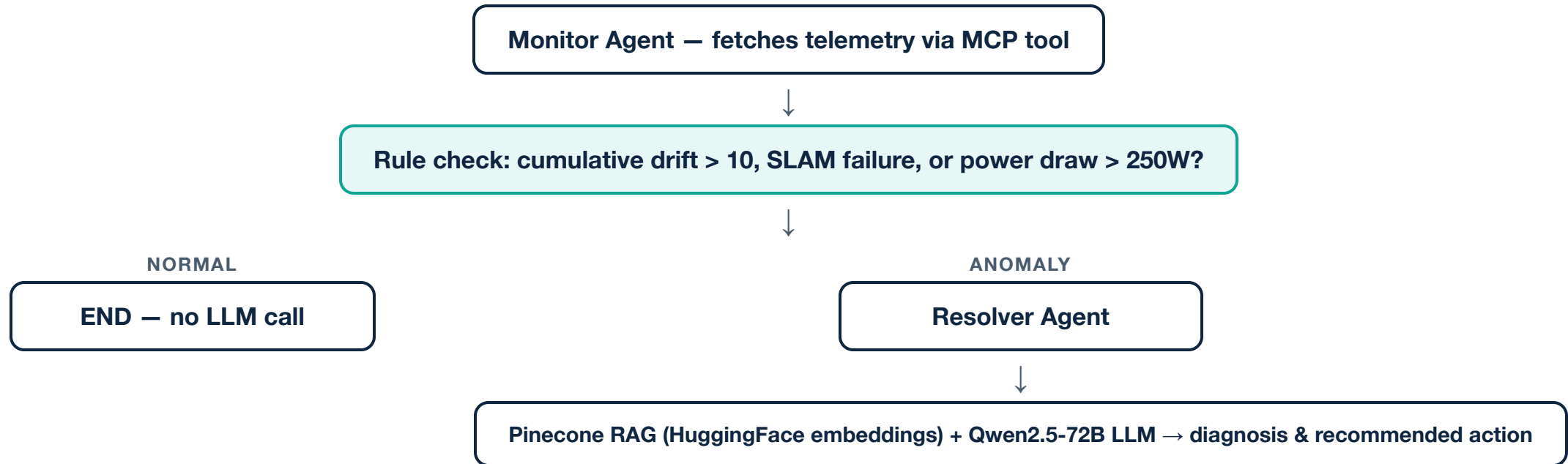
- Runs over the standard MCP **stdio** transport — any MCP-aware client or agent can connect without custom integration code.
- Decouples **what an agent can do** (call two well-defined tools) from **how the system works** (FastAPI internals stay hidden).
- Verified live: a full MCP client session listed both tools and received correct, structured results from each.

PHASE 3

Multi-Agent Workflow

A LangGraph graph of two agents — one that watches, one that diagnoses — connected by conditional routing so the LLM only runs when it's actually needed.

How an Anomaly Gets Diagnosed



A Diagnosis, End to End

A simulated high-drift anomaly (cumulative drift = 12.3) was pushed through the Resolver Agent. It correctly retrieved the matching SLAM documentation and produced a grounded diagnosis:

"Cumulative drift exceeding 10.0 indicates the SLAM module has lost tracking... Recommended action: trigger a re-localization routine or restart the SLAM node."

- Retrieval correctly matched the anomaly to the **ROS 2 SLAM Toolbox** reference document out of three candidate docs in the vector store.
- The LLM's recommended action **matched the source documentation** exactly — no hallucinated steps.

PHASE 4

Evaluation

LangSmith tracing and a 20-scenario test set measure whether the system detects the right anomalies — and diagnoses them for the right reasons.

Results Against 20 Test Scenarios

100%

Anomaly classification accuracy

20

Scenarios: normal, power, drift, SLAM failure

0

LLM calls on normal telemetry

- Anomaly flagging is **deterministic threshold logic**, so it's cheap, fast, and 100% reliable — no LLM judgment call needed.
- LangGraph's conditional edges route normal telemetry straight to **END**, skipping the LLM entirely and saving cost and latency.
- RAG relevancy held up across scenarios: retrieved docs matched the anomaly type in every anomalous case.

FINDINGS

Bottlenecks & Optimization Ideas

Where the time goes

- Resolver LLM generation: **1.5–3s** per anomalous run — the dominant cost
- Embedding the query before the vector search: **~200–400ms**
- MCP round-trip over HTTP: minor added network overhead

How to cut cost & latency

- Send only the specific sub-metric that triggered the anomaly, not the full telemetry object
- Reduce retrieved docs from k=2 to **k=1** — halves context with no quality loss
- Cache diagnoses for recurring anomalies (e.g. repeated SLAM failures) to skip the LLM entirely

Technology Stack

Layer	Technology
API Service	FastAPI, Pydantic, Uvicorn
Tool Access	Model Context Protocol (MCP), stdio transport
Orchestration	LangGraph — stateful multi-agent graph with conditional routing
Retrieval	Pinecone vector database + HuggingFace sentence embeddings (all-MiniLM-L6-v2)
Language Model	Qwen2.5-72B-Instruct, served via HuggingFace Inference Endpoint
Observability	LangSmith — tracing, token usage, latency, evaluator scoring
Testing	Pytest with FastAPI's TestClient

THANK YOU

Questions & Discussion

Source code, tests, and full documentation:

github.com/gowthamde24/project_sentinel